Motor ECU 개발 경험
기술면접 준비 노트

설계 이유 · 데이터 흐름 · 장애 진단 · 검증 한계 2026년 10월 17일 기준

60초 설명 예시

말하기 초안

STM32 기반 CAN-LIN 프로젝트에서 Motor ECU의 CAN 제어와 RTOS 구조를 다뤘습니다. VCU의 0x100 명령을 받아 Virtual Motor의 target RPM을 계산하고, DC 네 개와 SG90 두 개의 보조 출력으로 연결했습니다. CMSIS-RTOS2로 전환하며 수신·제어·출력·송신·상태 감시를 분리하고, 공유 상태를 mutex로 보호했습니다. 최신 Gateway 문서와 비교할 때 조향각과 Body 명령 후보의 Byte3 중복을 발견해 Byte4~5로 옮겼습니다. 전체 ARM 빌드와 모의 회귀 테스트를 통과했으며, 실물 지연 측정과 CPU fault·bus-off 복구는 남은 과제로 구분했습니다.

3분으로 늘릴 때의 순서

요구 30초: CAN 수신, DC4/SG90 2 출력, Virtual Motor baseline.
구조 45초: ISR → RX queue → 상태 mutex → 제어/출력, 별도 TX queue.
문제해결 60초: TX/RX 관측점으로 통신과 전력을 분리하고 DBC 충돌을 재현.
검증 30초: ARM 빌드, C/RTOS mock, Python15 tests.
한계 15초: 실제 최악 지연, CPU fault, bus-off는 미완료.

구분해서 말해야 할 네 가지

1. CMSIS-RTOS2는 API 계층이고, FreeRTOS 커널은 V10.3.1이다.
2. PWM 요청 비율은 실제 차량 속도나 실측 모터 RPM이 아니다.
3. 조향각은 논리 target이며 실측 SG90 축 각도나 바퀴 각도가 아니다.
4. mock 통과는 커널 선점·전기적 신호·실물 동작의 증명이 아니다.

01. 구조를 화이트보드로 설명하기

공유 데이터와 하드웨어 자원의 소유권부터 그린다.

데이터 흐름

CAN RX ISR → 캡처 시각을 포함한 RX queue → canRxTask → motorStateMutex로 보호한 runtime → control/actuator task → PWM

control task → TX queue → canTxTask → canTxMutex → HAL CAN mailbox

Task	우선순위 / 주기	책임
actuatorTask	High1(41) / 1ms	DC·서보 PWM target 적용, 자체 freshness 검사
motorControlTask	High(40) / 1ms	watchdog, 가상 모델, 고장 상태; 10ms 상태 생성
canRxTask	AboveNormal(32) / queue 대기	명령 decode, 캡처 시각·Alive 검증
canTxTask	Normal(24) / queue 대기	상태 송신; 20ms보다 오래된 대기 상태 폐기
healthTask	BelowNormal(16) / 500ms	LED, task stack 여유 관측

왜 출력 task와 제어 task를 나눴나?

모델 계산·상태 송신과 물리 출력 적용의 책임을 분리했다. 출력 task도 watchdog을 검사하므로 모델 갱신 지연에만 의존하지 않는다. 단, 같은 CPU/커널이 멈추면 둘 다 실행되지 않으므로 독립적인 하드웨어 안전 장치는 별도 문제다.

1ms task가 PWM을 1ms마다 토글하나?

아니다. task는 CCR duty/펄스 폭을 갱신한다. DC 파형은 TIM1/3/4의 20kHz, 서보는 TIM2의 50Hz 하드웨어 타이머가 생성한다. task 주기와 파형 주파수를 구분해야 한다.

핀을 설명한다면

CAN PA11/12, DC PA8/9·PB4/5·PB0/1·PB6/7, 서보 PA0/1. MDD3A의 제어 입력과 모터 출력 단자는 이름이 같아도 구분한다. 서보는 외부5V, SN65HVD230은3.3V, GND는 공통이며 모터 전압은 모터 정격에 따른다.

02. CAN·DBC 질문과 바이트 해석

최신 규격: DBC0.7 / firmware revision4 / script1.5.

0x100 VCU_Command	데이터 형식	의미
Byte0~1	unsigned16, Intel, 0.1Nm	TorqueRequest 0~4000Nm
Byte2 bit0~2 / bit3 / bit4~7	Gear / Brake / Alive	0P 1N 2D 3R / bool / 0~15 wrap
Byte3	Motor는 무시, script는0	Gateway Body 명령 후보용 확보
Byte4~5	signed16, Intel, 0.1도	SteeringAngle -45~45도
Byte6~7	Reserved, script는0	현재 미사용

예제 | 전진25%, 좌20도, Alive0

10 27 02 00 38 FF 00 00

데모 맵에서 25%는 TorqueRequest1000Nm → raw10000=0x2710 → 10 27. Gear D=2. 좌20도는 raw -200의 16bit 2의 보수0xFF38 → 38 FF. 조향은 Byte4~5에 들어간다.

왜 Byte3를 비웠나?

Gateway Routing의 Body Command 후보가 Byte3의 bit24~31을 사용했다. 옛 조향 -20도의 Byte3=0x38은 그 후보에서 Hazard=1, DoorLock=3으로 읽힌다. 후보가 공식 DBC에 아직 없더라도 통합 전에 겹침을 제거해야 했다. 이동 후 공식/시험 DBC와 디코더·송신기·테스트를 함께 바꿨다.

0x110 응답은 무엇인가?

Byte0~1 Virtual RPM, Byte2~3 TorqueApplied, Byte4의 low2bits MotorState와 다음4bits Alive이다. 확장 Byte5~6은 target echo, Byte7 low4bits는 fault flags, high4bits는 firmware revision이다. echo는 위치 센서 피드백이 아니다.

DBC와 펌웨어 버전은 왜 함께 바꾸나?

같은 CAN ID/DLC여도 바이트 의미가 바뀌면 조용한 오해석이 생긴다. script는 revision4만 구동을 허용한다. 다만 CAN command 자체에 송신기 버전 필드가 없으므로 Raw TX나 다른 VCU까지 자동 보호되는 것은 아니다. 모든 송신기의 규격을 맞춰야 한다.

03. 막힌 부분을 어떻게 좁혔나?

관측 → 가설 → 확인 → 남은 한계를 한 묶음으로 답한다.

관측 / 문제	원인 분리와 처리	면접에서 피할 단정
TX 증가, RX0	interface·500kbps·Listen-only·실행 상태·트랜시버 배선을 분리. 사용자 RX/TX 착오 수정 후 READY/fresh 응답 관찰	TX 카운터만으로 CAN 전달 성공 주장
명령은 보이나 모터 정지	CAN 수신과 드라이버 전력, PWM 핀, A/B 연결, 낮은 duty 기동 특성을 나눠 확인	통신 정상이라 모터 전력도 정상이라고 추정
서보가 안 움직임	초기 script에 mapping deferred 표기가 있었음. 이후 실제 TIM2 출력과 진단 echo를 추가한 revision 적용	초기 미구현과 이후 FAIL_SAFE를 같은 원인으로 단정
FAIL_SAFE 반복	timeout/Alive/invalid/RX loss flags를 구분하고 명령 나이·다중0x100 송신 여부를 관측	Alive 검사를 끄면 해결됐다고 주장
서보 회전량 작음	±45도 요청이1250~1750µs로 제한된 현재 변환 확인. 2배 확대는 제안만 했으며 적용하지 않음	실제 축 각도가 요청 각도와 같다고 주장
auto 입력 즉시 ValueError	interface 번호를 int로 받는 단계에 auto 입력. 먼저256 선택 후 motor 프롬프트에서 auto	이 오류를 CAN 버스 고장으로 해석

통신·논리·물리 출력의 관측점

PC 송신 요청 → 버스의 0x100 → ECU RX/Alive/age → target/duty/CCR → 핀 파형 → 드라이버 출력 → 실제 회전. 어느 지점까지 확인했는지 말하고 그 다음 지점을 측정한다. TX API 호출 수와 실물 전달률, Virtual RPM과 실제 RPM을 혼용하지 않는다.

자동 시퀀스는 무엇을 검증하는가?

30초 동안 PWM20~35%, 조향0/±20/±40도, 전진·정지·후진을 반복한다. s나 수동 명령이 자동 모드를 해제하며 상태 끊김/OFF/FAIL_SAFE/버전 불일치 후 자동 재개하지 않는다. 반복 명령 생성과 중단 로직을 검증하지만 실제 출력의 완전한 HIL 시험은 아니다.

04. RTOS·고장 처리 심화 질문

답을 암기하기보다 코드의 경계와 미완료 사항을 이해한다.

Q1. ISR에서 mutex를 잡지 않는 이유는?

mutex는 task 소유권과 대기를 전제로 한다. ISR은 timeout0 message queue put으로 캡처 프레임을 넘기고, task가 mutex로 상태를 갱신한다. CAN IRQ priority5도 RTOS syscall 허용 범위에 맞췄다.

Q2. 우선순위 역전은 어떻게 다뤘나?

motorStateMutex와 canTxMutex에 priority inheritance를 사용한다. 단, 상속만으로 deadline이 보장되지는 않는다. 임계구역 길이·블로킹 상한과 실물 부하를 측정해야 한다. state mutex는 최대2 ticks 획득 대기하며 출력 task 실패 시 SafeStop으로 간다.

Q3. 큐에서 꺼낸 시각을 수신 시각으로 쓰면?

오래 대기한 명령이 새 명령처럼 보일 수 있다. ISR 캡처 시각을 보존해 freshness를 판단한다. RX queue loss는 epoch로 추적하고 손실 전 대기 프레임을 버린 뒤 Alive를 재동기화한다.

Q4. timeout과 복구는 정확히 어떻게 동작하나?

수신 age가30ms를 초과하면 torque/targetRPM을0으로 만들고 Virtual RPM은 감쇠한다. 정상 task 경로에서 서보 target은 중앙이다. timeout 후 첫 프레임은 Alive seed로 쓰고 이후 연속 증가3회, 실제4프레임에서 해제된다. 문서의 "3회 확인"과 계수 기준 정렬은 아직 남아 있다.

Q5. bus-off가 되면 자동으로 정상화되나?

현재는 아니다. AutoBusOff가 비활성이고 별도 재시작 상태 머신도 없다. 수신이 끊기면 timeout 정지는 하지만 통신 복구까지 완성한 것은 아니다. ABOM 또는 관리된 재시작과 Alive 재동기화 정책을 정하고 주입 시험해야 한다.

Q6. HardFault에서도 출력 task가 안전하게 멈추나?

현재 예외 핸들러는 무한 대기하고 타이머는 마지막 PWM을 유지할 수 있다. 따라서 task 기반 timeout만으로 CPU fault를 보장하지 못한다. RTOS에 의존하지 않는 출력 차단과 독립 watchdog/하드웨어 차단을 설계할 필요가 있다.

05. 검증 질문·시연·마지막 점검

"테스트를 했다"보다 어떤 실패를 구분했는지 설명한다.

Q7. 어떤 테스트를 통과했나?

C 모델 회귀는 DBC, 제동/P/N, timeout, invalid, Alive wrap/연속성, 방향 반전 등을 검사했다. RTOS mock은 실제 task 함수의 큐·mutex 경계, 획득 실패 정지, 출력 task의 timeout, 큐 손실 복구를 검사했다. Python15 tests는 바이트 벡터, DBC 동일성/중복, firmware mismatch, 자동 순환·정지·고장 후 미재시작을 포함한다. revision4 ARM 전체 빌드도 통과했다.

Q8. 테스트 통과 후에도 왜 완료라고 말하지 않나?

mock은 실시간 커널 선점·실제 CAN 전달·전압 강하·타이머 파형을 재현하지 않는다. 실물 최대 지연, jitter, stack 여유, 네 모터 동시 부하, CPU fault·bus-off 복구는 별도 증거가 필요하다. 이번 작업에서 성능 향상률이나 실물 오류율을 측정하지 않았다.

Q9. AI는 어디에 사용했고 본인은 무엇을 했나?

코드 변경·테스트 작성·문서 초안을 AI 보조로 진행했다. 요구를 구체화하고 배선/실행 로그를 제공했으며 RX/TX 착오를 확인한 과정을 설명할 수 있다. 면접에서는 본인이 실제로 판단·수정·재현한 부분을 더해 구분해서 답한다. 작성하지 않은 코드를 혼자 작성했다고 주장하지 않는다.

짧은 시연 순서

1. 바퀴를 띄우고 서보 가동 범위·외부 전원·GND를 확인한다.
2. firmware4 실행, CANgaroo COM4/500kbps/normal 측정, script1.5 Load.
3. interface 프롬프트에서256 선택 → motor 프롬프트에서 status 확인.
4. READY/fresh/revision4 확인 후 낮은 출력·개별 조향 시험, 이후 auto.
5. s로 DC0/P/중앙, quit 후 외부 전원 종료. 고장 주입은 별도 계획으로 진행.

면접 전 직접 답해 볼 체크리스트

☐ -200을 38 FF로 설명할 수 있는가? ☐ Byte3 후보와 Byte4~5를 구분하는가?
☐ mutex와 queue의 역할이 다른 이유를 설명하는가?
☐ 설정 주기와 실측 지연을 구분하는가? ☐ 남은 결함 세 가지를 숨기지 않는가?
☐ 실제로 확인한 보드 동작과 팀 내 본인 역할을 구체적으로 말할 수 있는가?

근거: 저장소의 RTOS2_ARCHITECTURE.md, DEVELOP_ARCHITECTURE_REVIEW.md, MOTOR_STEERING_INTERFACE.md, MOTOR_ECU_PINOUT.md 및 C/Python 테스트·빌드 로그와 대화. 최신 검토 기준 develop226e88e. 역할·성과 수치는 사실 확인 후 개인화한다.